

Week 12 • Time-tracking, and the idea of context

Technical Interaction Design • ITU • Autumn 2026

Built 4 September 2026 • commit 4e11e8b

Latest version: <https://itu-tid.github.io/lecture-notes-pdf/Week-12.pdf>

CONTENTS

Authorization (in Parse)

- Reading
- Further reading

Authorization (in Parse)

Source: [Lectures/Technical/Backend/Authorization-and-ACL-in-Parse.md](#) — something unclear? [Open an issue](#), or send a pull request.

Motivation

Why can you not keep the Parse API keys perfectly secret?

- Remember the architectural diagram from the beginning of the course? bundle.js is sent to the browser...
- The JavaScript code of your web application can be inspected by another web programmer.

What happens if I access your repository and find your AppID and JSKey?

- Read info that is not meant for me
- Delete useful information
- Store my movie collection in your tables
- etc.

That is - **only if you have made your tables public**

- When we created the DB we were asked about access control and we agreed to make everything public because we're working on an MVP.
- Now it's time to harden the security of our database

What can we do if the API keys can't be made secret?

Use access control in such a way that even with the keys, no harm can be done

1. Limit access to tables
2. Limit access to individual objects
3. Restrict class creation

Let us take each of these in turn.

1. Limiting Access to Tables

For every table you can choose who has access to it and what privileges they have

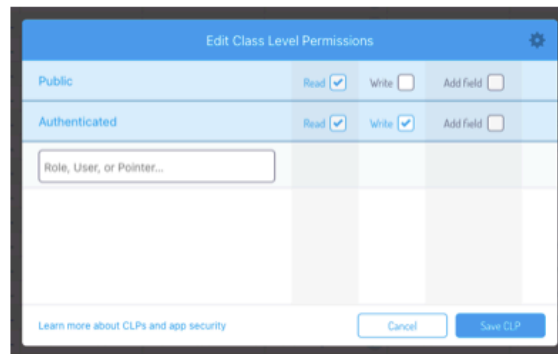
Who has access can be specified with multiple levels of granularity

- Public (anyone, even unauthenticated)
- Authenticated users (requiresAuthentication)
- Specific users
- Roles

Privileges for the entire class

- Get - retrieve individual objects by ID
- Find - query for objects
- Create - create new objects
- Update - modify existing objects
- Delete - delete objects
- Add Fields - add new fields to the schema

The image below shows the Parse UI for setting Class-level permissions



Practical Implication: For your applications, you can prevent non-authenticated users to access your tables

Obs: mappings from OO lingo to DB lingo

- table = class
- row = object

2. Object-Level Permissions

Even if now you only allow logged in users, it would still not be desirable that an unfriendly user creates an account and then

- reads other users data
- or even starts deleting other people's data!

This is where the **Access Control Lists** concept come into play. They allow you to set **fine-grained permissions** for every row in your table. Usually they are **created at the same time** as the object.

Who can ACL permissions apply to?

- Public
- Specific users
- Roles

Privileges per object

- Read - can retrieve/query this object
- Write - can update or delete this object

Examples

User creates a private note

In the following example, a logged in user, creates a private note and ensures that it is only himself that can access that note:

```
const Counter = Parse.Object.extend("Counter");
const privateCounter = new Counter();
privateCounter.set("name", "Times Checked Twitter");
privateCounter.set("count", "42");
privateCounter.setACL(
  new Parse.ACL(Parse.User.current());
privateCounter.save();
```

Read for public but write only for owner

It is sometimes desirable that an object can be **read by other users**, but just **can not be written by them**. For such a case the `Parse.ACL` object offers the `setPublicReadAccess(true)` method call:

```
const Post = Parse.Object.extend("Post");
const publicPost = new Post();
publicPost.set("content", "I love technical interaction design");
publicPost.setACL(new Parse.ACL(Parse.User.current()));
publicPost.setPublicReadAccess(true);
publicPost.save();
```

Access control lists can be modified every time an object is saved.

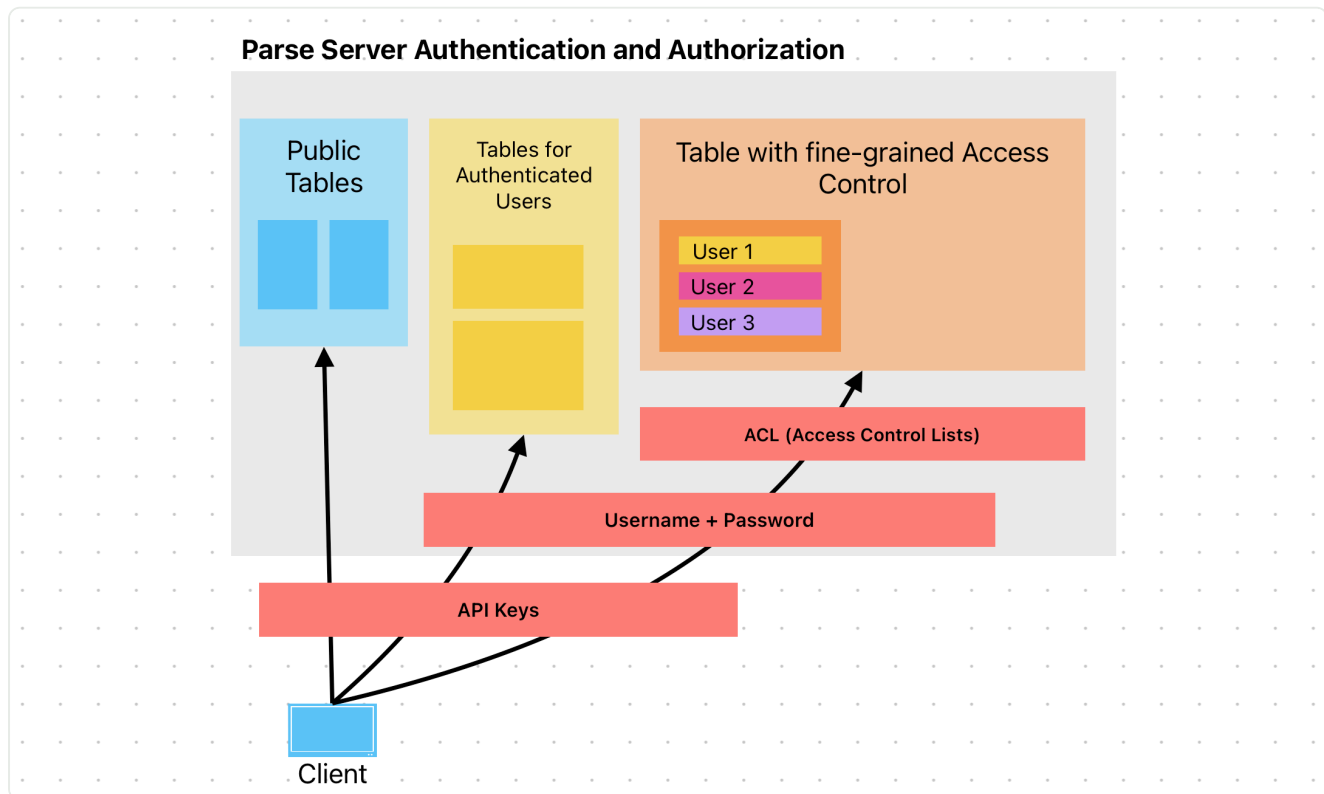
3. Restricting Class Creation

Surely, not all users should be allowed to create classes either!

Under `App Settings > Server Settings > Client Class Creation` you can specify if you expect users to be allowed to create new classes in your database. Probably you do not want that.

4. Combining Authorization Methods to Harden the Security of an Application

The methods above should be combined together to strengthen the DB access for your application.



In practice, there's no real reason to have any public tables. If it's a public list of objects, they can be hardcoded in the application.

Case Study: ToDo25

The simplest possible DB Model

```
erDiagram
    _User ||--o{ TodoItem : owns
    _User {
        objectId string PK
        username string
        email string
    }
    TodoItem {
        objectId string PK
        name string
        category string
        userId pointer FK
    }
```

One table and one pointer. Parse adds `objectId`, `createdAt` and `updatedAt` to every class for free, so the only fields we declare are the three that mean something. Note that there is no `List` here: a to-do belongs directly to a user, which is as small as the model can get — and it is the thing the sharing layer has to change.

an important field `userId`:

```
item.set("userId", Parse.User.current());
```

Why is this critical?

- Allows querying: "Show me only MY todos"
- Provides data privacy
- Enables access control

Define constants for our category values

```
export const CATEGORIES = {  
  IMPORTANT_URGENT: "Important & Urgent",  
  IMPORTANT_NOT_URGENT: "Important & Not Urgent",  
  NOT_IMPORTANT_URGENT: "Not Important & Urgent",  
  NOT_IMPORTANT_NOT_URGENT: "Not Important & Not Urgent"  
};
```

For the future, and a more flexible app, it would be nice to allow users to define their own categories.

Creating a service layer for the functions that interact with the DB

Why a Service Layer?

Instead of calling Parse directly from our UI components, we create a **service layer**:

Without a Service Layer code is more of a mess

- Components do not respect the **Single Responsibility Principle**: rendering the ui and talking to the DB
- Larger more complicated components
- Duplicated code

```
// In component - messy, duplicated code  
const TodoItem = Parse.Object.extend("TodoItem");  
  
const item = new TodoItem();  
item.set("name", name);  
item.set("category", category);  
item.set("userId", Parse.User.current());  
await item.save();
```

With a service layer code is simpler in the component

```
// In component - clean, simple
await createTodoItem(name, category);
```

Service layer folder

We have two files at the moment:

- `src/services/auth.js` - authentication functions
- `src/services/todoService.js` - handling of todo items

Defining the TodoItem related services

```
const TodoItem = Parse.Object.extend("TodoItem");
```

Helper function to convert Parse Todo object to plain JS object

- React works better with plain JS objects so it's nicer if we convert
- Also, we unify the treatment of the `id` and the other fields

```
function todoItemToPlainObject(parseObj) {
  return {
    id: parseObj.id,
    name: parseObj.get("name"),
    done: parseObj.get("done"),
    totalTime: parseObj.get("totalTime") || 0,
    currentSessionStart: parseObj.get("currentSessionStart"),
    category: parseObj.get("category"),
    createdAt: parseObj.get("createdAt"),
    updatedAt: parseObj.get("updatedAt"),
  };
}
```

Fetching items for user and category

- Note the multiple query conditions
- Note the ordering constraint


```
export async function fetchTodosByCategory(category) {
  const currentUser = Parse.User.current();

  const query = new Parse.Query(TodoItem);
  query.equalTo("category", category);
  query.equalTo("userId", currentUser);

  // Oldest first
  query.ascending("createdAt");

  try {
    const results = await query.find();
    return results.map(todoItemToPlainObject);
  } catch (error) {
    console.error("Error fetching todos:", error);
    throw error;
  }
}
```

But there's a problem: A malicious user could use Parse's REST API directly or modify the client code to read other users' todos! The query filter is just client-side convenience—it doesn't enforce security.

Access Controls when creating a new Todo item

```
export const createTodoItem = async (name, category) => {

  // Although it's not essential, we're using the getCurrentUser that we've defined in the
  // auth.js
  const currentUser = getCurrentUser();

  const item = new TodoItem();
  item.set("name", name);
  item.set("category", category);
  item.set("done", false);
  item.set("totalTime", 0);
  item.set("userId", currentUser); // ← Links to user!

  // Set ACL so only creator can read and write
  const acl = new Parse.ACL();
  acl.setReadAccess(currentUser, true);
  acl.setWriteAccess(currentUser, true);
  item.setACL(acl);

  try {
    const result = await item.save();

    // Return plain JavaScript object
    return {
      id: result.id,
      name: result.get("name"),
      done: result.get("done"),
      totalTime: result.get("totalTime"),
      currentSessionStart: result.get("currentSessionStart"),
      category: result.get("category"),
      createdAt: result.get("createdAt"),
      updatedAt: result.get("updatedAt")
    };
  } catch (error) {
    console.error("Error creating todo:", error);
    throw error;
  }
};
```

Observation: A more concise way

```
const acl = new Parse.ACL();
acl.setReadAccess(currentUser, true);
acl.setWriteAccess(currentUser, true);
item.setACL(acl);
```

// is equivalent to

```
const acl = new Parse.ACL(currentUser);
item.setACL(acl);
```

Other object-level configurations

Sharing with another user

```
const acl = new Parse.ACL(currentUser); // owner has full access
acl.setReadAccess(otherUser, true); // other user can read
acl.setWriteAccess(otherUser, true); // other user can write
```

Public read - owner write

```
const acl = new Parse.ACL(currentUser); // owner has full access
acl.setPublicReadAccess(true); // anyone can read
```

Role-Based Access

```
const acl = new Parse.ACL(currentUser);
acl.setRoleReadAccess("TeamMembers", true);
acl.setRoleWriteAccess("TeamMembers", true);
```

Two types of roles

Application-level roles (Moderators, Admins, Premium Users)

- Created manually in Parse Dashboard or via Cloud Code
- Managed by the app administrators, not end users
- Public read of the role is normal - users should see who the moderators are

User-created roles (e.g. Family, MyTeam, ProjectX)

- Created programmatically by regular users from the app
- Each user manages their own teams
- Private - no reason for others to see them

```
// Say user creates a "TeamMembers" role

const roleACL = new Parse.ACL(currentUser);

const role = new Parse.Role("TeamMembers", roleACL);

// Later our user can add other users to this role
role.getUsers().add(user1);

await role.save();
```

ACL does not work at the field level

In Parse, **ACLs work at the object level, not at the field/column level**. This means you cannot make some fields public and other fields private within the same object. **Example scenario** - You want everyone to **see your todo task name** and done status (public read) - But you want to **keep your time tracking data private** (only you can read)

Problem: You can not set some fields private and some public

```
// This does NOT work - you can't set different permissions per field
const todo = new TodoItem();
todo.set("name", "Write report");           // want this PUBLIC
todo.set("done", false);                    // want this PUBLIC
todo.set("totalTime", 3600000);              // want this PRIVATE
todo.set("currentSessionStart", new Date()); // want this PRIVATE

const acl = new Parse.ACL(currentUser);
acl.setPublicReadAccess(true); // This makes ALL fields public!
todo.setACL(acl);
```

Solution: Split data into two tables with a 1-to-1 relationships

Table 1: TodoItem (Public)

```

const TodoItem = Parse.Object.extend("TodoItem");
const todo = new TodoItem();
todo.set("name", "Write report");
todo.set("done", false);
todo.set("userId", currentUser);

// Public read, owner write
const acl = new Parse.ACL(currentUser);
acl.setPublicReadAccess(true);
todo.setACL(acl);
await todo.save();

```

Table 2: TodoTimeTracking (Private)

```

const TodoTimeTracking = Parse.Object.extend("TodoTimeTracking");

const timeTracking = new TodoTimeTracking();

timeTracking.set("todoId", todo); // Pointer to the TodoItem
timeTracking.set("totalTime", 3600000);
timeTracking.set("currentSessionStart", new Date());
timeTracking.set("userId", currentUser);

// Private - only owner can read and write
const acl = new Parse.ACL(currentUser);
timeTracking.setACL(acl);
await timeTracking.save();

```

Folder Structure

Separate Pages and Reusable Components

My favorite way of organizing

- **pages** – One file per page/view
- **components** – Reusable UI components
- **services** – API/backend logic
- **utilities** – a catch all for things that we don't know where to put yet

```
src/  
├─ assets/  
├─ pages/  
│   ├─ LoginPage.jsx  
│   └─ HomePage.jsx  
├─ components/  
│   ├─ TodoList/  
│   │   ├─ TodoList.jsx  
│   │   └─ TodoItem.jsx  
├─ services/  
│   ├─ authService.js  
│   └─ todoService.js  
├─ constants/  
├─ utilities/  
└─ App.jsx
```

Also, refactor, refactor, refactor. When you find a better organization, go with that.

Reading

From the ParsePlatform.org Guide:

- [Class-level Permissions](#)
- [Object-level Access Control](#)

Something unclear in this section? Open an issue.

Further reading

- SOLID principles
 - **Single Responsibility Principle** -
 - Open-closed Principle -
 - Liskov Substitution -
 - Interface Segregation -
 - **Dependency Injection Principle** -

Exam Questions

1. Why can't Parse API keys be kept secret in a web application?
2. What is the difference between authentication and authorization?
3. What are the three methods of access control in Parse?
4. What is an ACL and at what level does it operate?
5. Explain what this code does:

```
const privateNote = new Note();
privateNote.set("content", "My secret");
privateNote.setACL(new Parse.ACL(Parse.User.current()));
await privateNote.save();
```

6. How would you make an object publicly readable but only writable by the owner?
7. What's the problem with this code if we only validate name length on the client?

```
// Client-side validation
if (name.length > 200) {
  alert("Name too long!");
  return;
}
await todoItem.save();
```

8. What are two benefits of placing this code in a separate service file rather than directly in a React component?

```
export async function fetchTodosByCategory(category) {
  const query = new Parse.Query(TodoItem);
  query.equalTo("category", category);
  query.equalTo("userId", Parse.User.current());
  const results = await query.find();
  return results.map(todoItemToPlainObject);
}
```

9. What are the two types of roles in Parse and how do they differ?
10. Why can't ACLs be set at the field level, and what's the workaround?

Something unclear in this section? Open an issue.